ECON 609: Parallelization

Minsu Chang

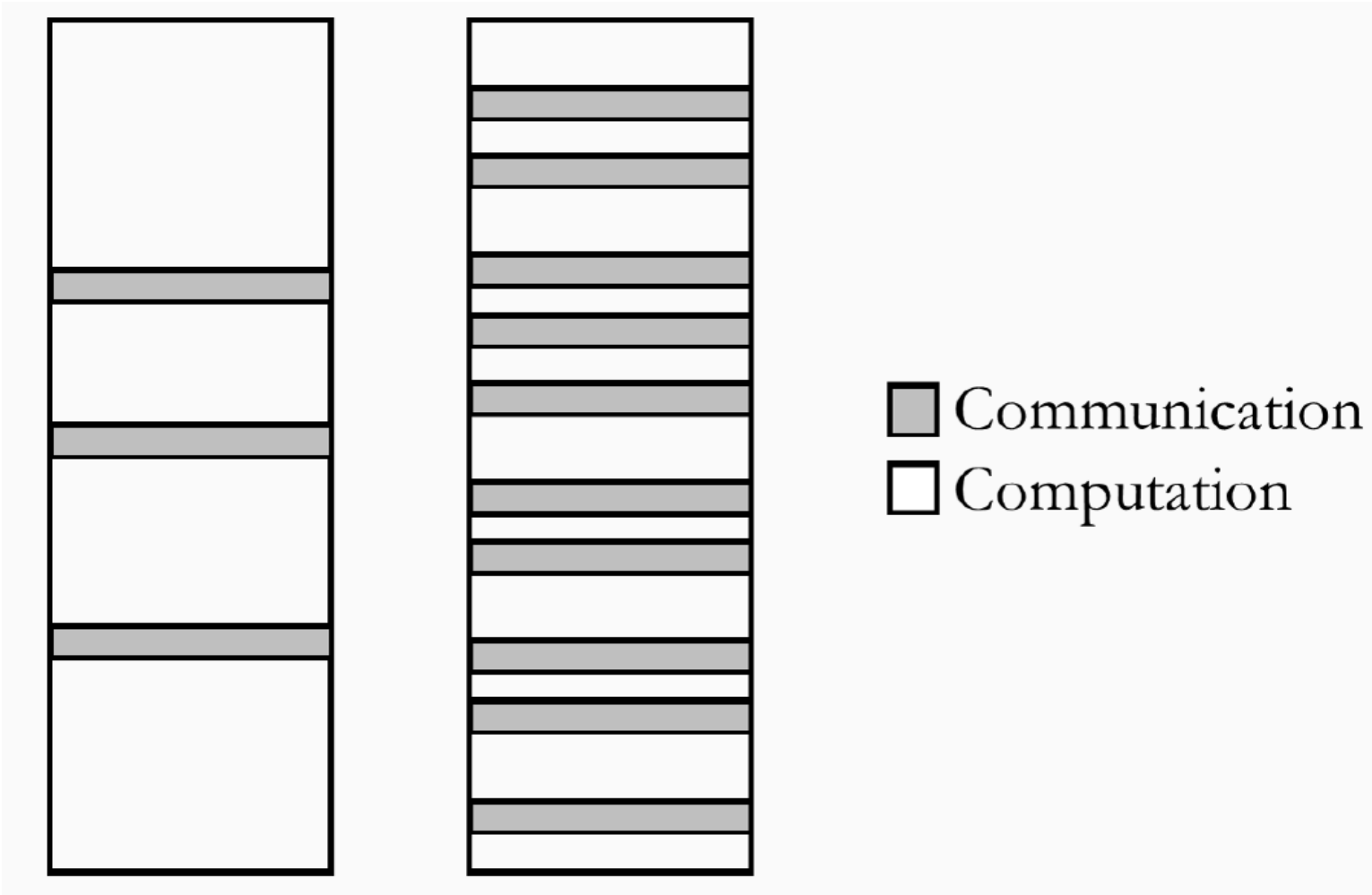
Georgetown University

Lecture # 9

Parallel Programming

- Main idea: divide a complex problem into easier parts.
- Scalability:
  (1) Strongly scalable: problems that are inherently easy to parallelize.
  (2) Weakly scalable: problems that are not.
- Granularity:
  (1) Coarse: more computation than communication
  (2) Fine: more communication
- Load balancing

Granularity



Some Comments

- Whether or not the problem is easy to parallelize may depend on the way you set it up.
- Taking advantage of your architecture.
- Trade off between speed up and coding time.
- Debugging and profiling may be challenging.

Example I: Value Function Iteration

$$V(k) = \max_{k'}\{u(c) + \beta V(k')\}$$

$$c = k^{\alpha} + (1-\delta)k - k'$$

(1) We have a grid of capital with 100 points, $k \in [k_1, k_2, \ldots, k_{100}]$.

(2) We have a current guess $V^n(k)$.

(3) We can send the problem:

$$\max_{k'}\{u(c) + \beta V^n(k')\}$$

$$c = k_1^{\alpha} + (1-\delta)k_1 - k'$$

to processor 1 to get $V^{n+1}(k_1)$.

(4) We can send similar problem for each k to each processor.

Example II: Random Walk Metropolis-Hastings

- Draw $\theta \sim P(\cdot)$.
- How?

  (1) Given a state of the chain θ_{n-1}, we generate a proposal:

$$\theta^* = \theta_{n-1} + \lambda\epsilon, \quad \epsilon \sim \mathcal{N}(0,1)$$

  (2) We compute: $\alpha = \min\left\{1, \frac{P(\theta^*)}{P(\theta_{n-1})}\right\}$.

  (3) We set:

$$\theta_n = \theta^*, \quad \text{w.p. } \alpha$$

$$\theta_n = \theta_{n-1}, \quad \text{w.p. } 1-\alpha$$

- Problem: to generate θ^*, we need to have θ_{n-1}.
- Parallel chains violate the asymptotic properties of the chain.

Example III: Life-Cycle Model

- Households solve:

$$V(t, e, x) = \max_{c, x'} \frac{c^{1-\sigma}}{1-\sigma} + \beta \mathbb{E} V(t+1, e', x')$$

$$s.t. \quad c + x' \leq (1+r)x + ew$$

$$P(e'|e) = \Gamma(e)$$

$$x' \geq 0$$

$$t \in \{1, \ldots, T\}$$

Computing the Model

(1) Choose grids for assets $X = \{x_1, \ldots, x_{n_x}\}$ and shocks $E = \{e_1, \ldots, e_{n_e}\}$.

(2) Backward induction:

- For $t = T$ and every $x_i \in X, e_j \in E$, solve the static problem:

$$V(t, e_j, x_i) = \max_c u(c) \quad s.t. \quad c \leq (1 + r)x_i + e_j w$$

- For $t = T - 1, \ldots, 1$, use $V(t + 1, e_j, x_i)$ to solve:

$$V(t, e_j, x_i) = \max_{c, x' \in X} u(c) + \beta \mathbb{E} V(t + 1, e', x')$$

$$s.t. \quad c + x' \leq (1 + r)x_i + e_j w$$

$$P(e'|e_j) = \Gamma(e_j)$$

In Parallel

(1) Set $t = T$.

(2) Given t, the computation of $V(t, e_j, x_i)$ is independent of the computation of $V(t, e_{j'}, x_{i'})$, for $i \neq i', j \neq j'$.

(3) One processor can compute $V(t, e_j, x_i)$ while another processor computes $V(t, e_{j'}, x_{i'})$.

(4) When the different processors are done at computing $V(t, e_j, x_i), \forall x_i \in X$ and $\forall e_j \in E$, set t to be $t - 1$.

(5) Go to (1).

- Note that the problem is NOT parallelizable on t. The computation of $V(t, e, x)$ depends on $V(t + 1, e, x)$.

MATLAB Code Layout

Using the `parallel toolbox`:

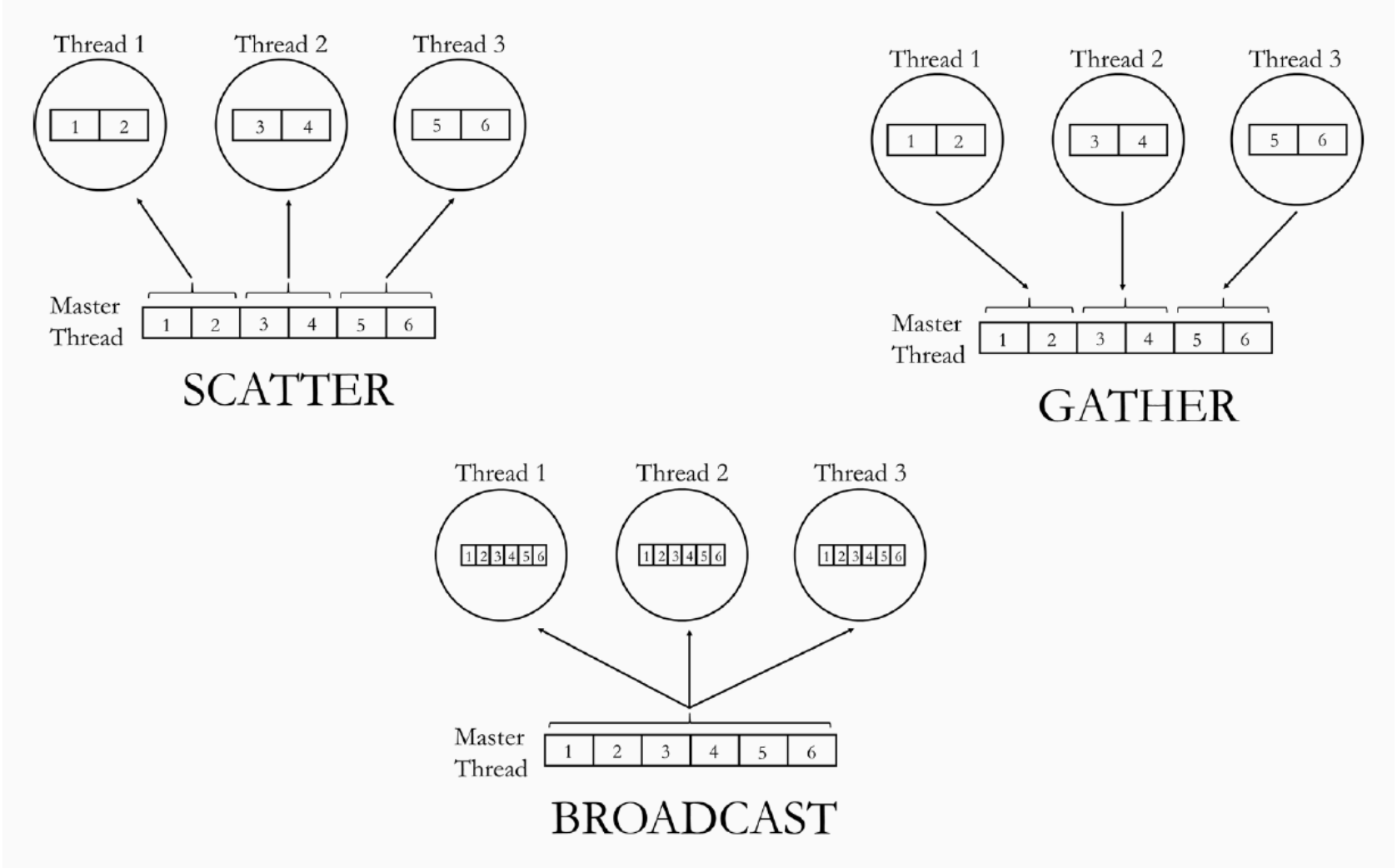
1. Initialize number of workers with `parpool()`:

```
parpool(6)
```

2. Replace the `for` loop with `parfor`:

```
for age = T:-1:1
    parfor ie = 1:1:ne
        % ...
    end
end
```

In Parallel


Thread 1
Thread 2
Thread 3
1 2
3 4
5 6
Master Thread
1 2 3 4 5 6
SCATTER
Thread 1
Thread 2
Thread 3
1 2
3 4
5 6
Master Thread
1 2 3 4 5 6
GATHER
Thread 1
Thread 2
Thread 3
1 2 3 4 5 6
Master Thread
1 2 3 4 5 6
BROADCAST

Costs of Parallelization

- Amdahl's law: the speedup of a program using multiple processors in parallel computing is limited by the time needed for the sequential fraction of the program.
- Costs:
  - Starting a thread or a process/worker.
  - Transferring shared data to worekrs.
  - Synchronizing.
- Adding more threads than cores available may even reduce performance.